UMHackathon 2026: Preliminary Round Deliverable

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

Domain: 2

Group Name: Palindrome

Group ID: 193

Project Name: SmartStock AI

Group Members

Name	Role	University
Maha Vaarshinie A/P Arupatha Rajoo	Team Leader	Multimedia University
Nareenthiran A/L Bhuvaneswaran	Team Member	Multimedia University
Yasha Durgashine	Team Member	Multimedia University

Table of Contents

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3

1.2 In-Scope Core Features (The MVP Focus) 3

1.3 Out-of-Scope (Strategic Exclusions) 4

2. Risk Assessment & Mitigation Strategy 5

3. Test Environment & Execution Strategy 7

4. CI/CD Release Thresholds & Automation Gates 9

4.1 Integration Thresholds (Merging to Main) 10

4.2 Deployment Thresholds (Pushing to Production) 11

5. Test Case Specifications (Drafts) 11

6. AI Output & Boundary Testing (Drafts) 17

6.1 Prompt / Response Test Pairs 18

6.2 Oversized / Large Input Testing 19

6.3 Adversarial / Edge Prompt Testing 20

6.4 Hallucination Handling 21

Implemented Mitigation Strategies: 21

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - Team Palindrome
Team Repo URL	https://github.com/NareenBruce/smartstock
Project Board URL	https://docs.google.com/spreadsheets/d/1Ezb55-MdUt-sBY2IrKNgHvJmbxegvZzZXiG6OW7fw0M/edit?usp=sharing

Objective:

Ensure SmartStock AI provides reliable inventory forecasts by fusing structured sales data and unstructured notes using the ILMU GLM-5.1, while maintaining 3NF data integrity and minimizing AI hallucinations.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

1.2 In-Scope Core Features (The MVP Focus)

To maintain high performance during the 10-day sprint, our testing focus is narrowed to the "Decision Intelligence" core:

- **Multi-Modal Ingestion & Validation:**
  - CSV file header validation (format correctness)
  - Natural language prompts sanitization against prompt injection
- **Agentic Reasoning Engine (ILMU GLM-5.1):**

- Verification of Summarization, Interpretation, and Generation pipeline steps.
- Validation of tradeoff analysis and recommendation of strategic pivot.

- **Explainable Dashboard & Alerting:**
  - Full chain from data ingestion until Pivot Notification generation.
  - Real-time logging of reasoning engine's actions for historical analysis.

1.3 Out-of-Scope (Strategic Exclusions)

The following use cases are out of scope for this testing process, which aims at ensuring the stability of the orchestration of reasoning:

- **IoT & Real-Time Sensing Systems:** Stock inventory tracking using physical sensors.
- **ML Models Testing:** Time series predictions (ARIMA, LSTM) as Z.AI GLM is required only.
- **Multi-user Support & Enterprise-level ERP Integration:** SAP / Oracle synchronization or ROL-based access controls

2. Risk Assessment & Mitigation Strategy

We utilize the 5 X 5 Risk Assessment Matrix, where the Risk Score = Likelihood × Severity. This ensures that critical failures, such as AI-driven stockout errors, are prioritized for extensive testing

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
AI Hallucination & Reasoning Error: The GLM suggests stock levels that contradict the 3-month sales facts in the CSV.	3	5	15 (High)	Enforce JSON Schema Validation for all outputs and implement a "Summarization Stage" to filter noisy data before reasoning.	**Adversarial Testing:** Input malformed CSV data to verify if the system flags discrepancies and blocks invalid insights.
Database Transaction Failure: The STOCK_MOVEMENT junction table fails to update, causing a "Black Hole" in inventory tracking.	2	5	10 (Medium)	Implement SQL Transaction Rollbacks (ACID compliance) within the FastAPI backend logic.	**Integration Testing:** Simulate a server crash during a SQL INSERT to verify that no partial data is persisted.

Context Window Overflow: Large CSV uploads (5,000+ rows) exceed the token limits of the ILMU GLM-5.1 API.	4	3	12 (High)	Apply automated Token Chunking and SKU-based batching in the Reasoning Orchestrator.	**Boundary Testing:** Use 6,000+ row files to confirm the system segments data without losing historical context.
UI Latency & API Timeout: The complex agentic reasoning process takes >5 seconds, leading to store manager frustration.	4	2	8 (Medium)	Use Asynchronous API calls with a frontend "loading state" and a 5s timeout fallback.	**NFR Performance Test:** Measure response time under 50 concurrent user requests using Postman/JMeter.

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	Risk Level	Recommended Action
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

3. Test Environment & Execution Strategy

- **Unit Test**
  - **Coverage:** Data Ingestion Validation and JSON Schema Compliance.
  - **Implementation:** Tests will be coded in PyTest; run locally during development and through the CI pipeline on every repository push.
  - **Isolation:** Exogenous factors, such as Amazon RDS (PostgreSQL) and Z.AI API, are mocked to test Reasoning Orchestrator logic in isolation.
  - **Pass Criteria:** Correct validation of "Happy Cases" (CSV data) and "Negative Cases" (wrong header) with proper error handling formats.

- **Integration Test**
  - **Coverage:** Connectivity between Inventory Service and Reasoning Service.
  - **Implementation:** After passing Unit Tests, integration tests are conducted during feature branch merges (for example, merge CSV parsing features with GLM reasoning).
  - **Process:** Confirms that API calls and database transactions with PostgreSQL are successful without mocking to ensure data persistence.

- **Pass Criteria:** Integration points such as "Context Fusion" to "Strategy Alerting" operate well and produce accurate reasoning outputs.

- **Test Environment & CI/CD Practices**
  - **Local Testing:** Developers test manually and automatically on localhost before pushing changes.
  - **Staging/CI (GitHub Actions):** Automated testing occurs with every commit; verifies there are no build issues.
  - **Pipeline Automation:** GitHub Actions perform linting locally, but PyTest is mandatory for all PRs into the Main branch.

- **Regression Testing & Pass/Fail Rules**
  - **Execution Phase:** Automated regression testing includes all three identified in-scope functionalities (Ingestion, Reasoning, and Alerting), performed whenever a new branch gets merged with the Main.
  - **Pass/Fail Criteria:** The test can be considered “Passed” only if the actual result completely matches the expected result, with all others being documented in Defect Log.
  - **Continue Criterion:** All critical initial test cases need to reach the threshold before any other steps are allowed.

- **Test Data Strategy**
  - **Manual Input Data:** 5 different mock users were created, with varying types of store data to allow for testing various types of inventory.
  - **Automated Input Data:** The use of SQL seeding scripts that load PostgreSQL 3NF with some dummy data regarding historical purchases to stress-test the GLM summarization step.

- **Passing Rate Threshold**
  - **Overall Passing Rate Threshold:** Not less than 90% of all executed test cases need to be passed for any non-critical functionality.
  - **Critical Passing Rate Threshold:** 100% passing rate required for all critical reasoning and SQL data persistence checks (such as Z.AI API response check).
  - **Reason:** Critical errors will cause financial losses directly in terms of spoiled product or shortage of the item in question.

4. CI/CD Release Thresholds & Automation Gates

Any code changes are integrated or deployed within the SmartStock system. The purpose of these thresholds is to ensure that every update maintains system stability, reliability, and performance, especially given the dependency on AI-generated outputs.

SmartStock adopts a CI/CD (Continuous Integration and Continuous Deployment) pipeline, where automated checks are triggered whenever new code is pushed to the repository. These checks act as automation gates, meaning that if any requirement is not met, the pipeline will automatically block further progression. This prevents unstable or faulty code from reaching the main branch or production environment.

By enforcing these thresholds, the system ensures:

- Early detection of bugs and errors
- Consistent code quality across the team
- Reliable AI output performance
- Reduced risk of system failures in production

4.1 Integration Thresholds (Merging to Main)

The main branch serves as the single source of truth for the SmartStock application and is expected to always remain stable, functional, and deployment-ready. For this reason, strict integration thresholds are enforced before any code can be merged into the main branch.

When a developer submits changes (e.g., via a pull request), the CI pipeline automatically runs a series of validation checks. These checks ensure that the new code integrates smoothly with existing components without introducing errors or inconsistencies.

Checks	Requirements	Project	Pass/Fail
Automatic Build	Zero build errors	5 Errors	Failed
Unit Tests	100% pass rate for core logic	100%	Passed
Code Quality	Minimal linting issues	97%	Passed
Test Coverage	Minimum 80% coverage	79%	Failed

Only when all these conditions are satisfied will the system allow the merge to proceed. If any check fails, the pipeline blocks the merge, and the developer must resolve the issues before resubmitting.

4.2 Deployment Thresholds (Pushing to Production)

Before deployment, additional checks ensure that both system performance and AI outputs meet required standards.

Checks	Requirements	Project	Pass/Fail
Regression Testing	Minimum 90% pass rate	91%	Passed
AI Output Pass Rate	Minimum 80% valid responses	95%	Passed
Critical Bugs	Zero critical (P0/P1) bugs	0	Passed
API Performance	Response time < 2–5 seconds	450 ms	Passed
Security	No exposed API keys	Clean	Passed

5. Test Case Specifications (Drafts)

This section defines a set of structured test cases used to validate both the **functional correctness** and **non-functional performance** of the SmartStock system. The purpose of these test cases is to ensure that the system behaves as expected under normal, edge, and stress conditions, while also verifying the reliability of the AI-driven decision-making component.

Each test case includes a clear description, step-by-step execution process, and expected outcomes to ensure consistency in testing and evaluation. The selected test cases cover critical user flows, including data input, AI processing, and system performance.

1. **TC-01: Happy Case (End-to-End Flow)**

Field	Details
Test Case ID	TC-01
Test Type	Happy Case (End-to-End Functional Flow)
Feature	Historical Data Ingestion & AI Reasoning
Description	Verify that an SME user can successfully upload a 7-day historical sales CSV, input unstructured context notes, and receive an optimized inventory purchasing strategy from the Z.AI GLM.
Test Steps	1. Open the SmartStock React dashboard in the browser. 2. Upload a valid 7-day sales CSV file (test_data.csv). 3. Input unstructured context note (e.g., "Severe storm incoming tomorrow, expect 50% traffic drop"). 4. Click the "Generate Decision Matrix" button. 5. The system validates the CSV format and enforces the 1500-token context window limit. 6. FastAPI backend constructs the prompt and queries the Z.AI GLM. 7. Frontend renders the "Predictive Trends" chart and the structured "AI Strategy Matrix".
Expected Result	The CSV data is successfully ingested into the SQLite database. The Z.AI GLM successfully returns a strictly formatted JSON response containing mock_strategy, mock_tradeoff, and a riskLevel score. The dashboard updates

	to proactively recommend cutting orders for highly perishable items with a high Risk Level score (>80).
Actual Result	The CSV was successfully parsed and stored without a 422 error. The Z.AI GLM returned the correctly structured JSON response within the required API timeout limits. The React frontend correctly visualized the 7-day predictive charts and the actionable decision cards.
Status	Passed

2. **TC-02: Negative / Edge Case (Invalid Input Handling)**

Field	Details
Test Case ID	TC-02
Test Type	Negative Case (Input Validation)
Feature	Adversarial Data Ingestion & Validation
Description	Verify the system handles invalid file types (e.g., .pdf, .docx) gracefully by blocking the upload and returning a 422 Unprocessable Entity error, preventing database corruption and application crashes
Test Steps	1. Open the SmartStock React dashboard in the browser.

	2. Attempt to upload an unsupported file type, such as System_Analysis.docx or PRD.pdf, instead of the required .csv historical sales file 3. Click the "Generate Decision Matrix" button. 4. Observe the frontend validation and UI error messages. 5. Bypass frontend validation using an API tool (e.g., Postman) to directly POST the invalid file to the /api/v1/upload-sales FastAPI backend endpoint
Expected Result	The frontend immediately blocks the upload based on the file extension check and displays a user-friendly error toast. When bypassed, the FastAPI backend rejects the multipart form data and returns a strict 422 Unprocessable Entity HTTP error. The SQLite database remains uncorrupted, and the Z.AI GLM inference is halted before triggering any 500 Internal Server Error
Actual Result	The system successfully identified missing fields and prompted the user to correct the input before proceeding.
Status	Passed

3. TC-03: NFR (Performance Test)

Field	Details
Test Case ID	TC-03

Test Type	Non-Functional Requirement (Performance)
Feature	API Response & Latency Test
Description	Verify that the FastAPI backend endpoint (/api/v1/analyze-inventory) remains responsive under concurrent user load and enforces the defined latency thresholds for the Z.AI GLM calls
Test Steps	1. Open an API testing tool like Postman 2. Target the /api/v1/analyze-inventory endpoint. 3. Send 50 concurrent POST requests simulating SME users submitting their CSV data and context notes 4. Record the average response time and track the error rate
Expected Result	The average API response time must remain under 800ms for internal processing. For the external Z.AI GLM inference, the system must strictly enforce a 5-second asynchronous timeout to prevent UI freezing. The system must maintain a 0% internal server error (500) rate.
Actual Result	The average FastAPI response time was successfully recorded at 450ms under load. The system maintained a 0% error rate, and no requests caused the backend to freeze or exceed the 5-second timeout threshold.
Status	Passed

4. **TC-04: NFR (Load Test / Scalability)**

Field	Details
Test Case ID	TC-04
Test Type	Non-Functional Requirement (Load & Scalability)
Feature	Containerized Backend & Tunnel Scalability
Description	Verify that the containerized FastAPI backend and Cloudflare Zero Trust Tunnel infrastructure can scale and process a sudden surge of concurrent SME users uploading historical inventory CSVs without failing.
Test Steps	1. Configure a load-testing script (e.g., Apache JMeter or Locust). 2. Target the public Cloudflare backend endpoint (https://api-smartstock.nareenbruce.tech/api/v1/upload-sales). 3. Simulate 100 concurrent SME users uploading the 7-day test_data.csv file simultaneously. 4. Monitor Docker container resource utilization (CPU/Memory) and track the HTTP response codes.
Expected Result	The FastAPI asynchronous workers successfully handle the concurrent file I/O operations without blocking the server. The Docker container resource usage remains within safe limits, and the SQLite database processes the concurrent inserts without throwing a database is locked operational error. The expected success rate is 100% (HTTP 200 OK).

Actual Result	The FastAPI backend successfully parsed and ingested all 100 concurrent CSV uploads using its asynchronous event loop. Docker container memory remained stable, and the Cloudflare tunnel successfully routed all traffic without rate-limiting or dropping connections. 0% Error Rate observed.
Status	Passed

6. AI Output & Boundary Testing (Drafts)

This section focuses on validating the behavior, reliability, and robustness of the Z.AI General Language Model (GLM) within the SmartStock system. Since the AI component is the core decision-making engine, it is critical to ensure that outputs are accurate, relevant, and safe under a variety of conditions.

The testing approach evaluates the AI across multiple dimensions, including normal usage scenarios, structured data handling, edge cases, adversarial inputs, and system boundary limits. Each test ensures that the model produces meaningful recommendations while minimizing risks such as hallucination, invalid outputs, or system failure.

6.1 Prompt / Response Test Pairs

This set of tests validates how the GLM responds to different types of inputs, ensuring that outputs are context-aware, logically consistent, and aligned with business needs.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	"Sales: 40 coffees/day, rain tomorrow. What should I do?"	The system should generate a recommendation based on both structured (sales) and contextual (weather) data. Output must include reasoning (e.g., reduced demand due to rain) and avoid hallucinated information.	Suggested reducing milk stock by approximately 20%, with explanation linking rain to lower customer traffic.	Passed
AI-02	Structured JSON sales data (multiple products and trends)	Output must accurately reflect trends from the provided dataset, with no fabricated values or assumptions. Recommendations should be data-driven and consistent with input.	Output aligned with input data trends, correctly identifying high- and low-performing items.	Passed
AI-03	"Festival next week, high demand expected"	The system should interpret the event as increased demand and recommend stock increases with clear justification.	Suggested increasing inventory by approximately 25%, with reasoning based on anticipated higher demand.	Passed

Evaluation Insight:

These tests confirm that the GLM can effectively combine structured and unstructured inputs to produce context-aware and actionable recommendations, which is essential for SmartStock's decision intelligence functionality.

6.2 Oversized / Large Input Testing

This test evaluates how the system handles inputs that exceed defined limits, ensuring that performance and output quality are maintained without exceeding token constraints.

Field	Details
Maximum Input Size	CSV: ~5,000 rows / Text: ~2,000 tokens
Input Used for Testing	CSV dataset containing approximately 10,000 rows
Expected Behavior	The system should not send the full dataset directly to the GLM. Instead, it should apply preprocessing techniques such as chunking and summarization to reduce input size while preserving key information.
Actual Behavior	The system successfully summarized the dataset into key trends and metrics before sending it to the GLM, ensuring efficient processing.
Status	Passed

Evaluation Insight:

This test demonstrates that SmartStock can handle large datasets efficiently by applying preprocessing strategies, ensuring that the system remains scalable and does not exceed model limitations.

6.3 Adversarial / Edge Prompt Testing

This test evaluates how the system handles unclear, insufficient, or potentially misleading user inputs. It ensures that the AI does not generate unreliable or unsafe recommendations when given inadequate data.

Field	Details
Prompt	"Give random stock advice without data"
Expected Behavior	The system should detect insufficient context and avoid generating arbitrary or misleading recommendations. It should return a safe fallback response indicating that more data is required.
Actual Behavior	The system responded with a fallback message: "Insufficient data for recommendation," preventing unreliable output.
Status	Passed

6.4 Hallucination Handling

Given that large language models may occasionally generate incorrect or fabricated outputs (hallucinations), SmartStock incorporates multiple strategies to detect, prevent, and handle such cases.

Implemented Mitigation Strategies:

- **JSON Schema Validation**
  All AI responses are validated against a predefined structure to ensure outputs are complete, consistent, and correctly formatted.
- **Retry Mechanism**
  If an output is invalid or inconsistent, the system automatically retries the request with a refined prompt to improve accuracy.
- **Prompt Constraints**
  Prompts are carefully designed to limit ambiguity and guide the model toward relevant, data-driven responses.
- **Fallback Response Mechanism**
  When the system detects low confidence or insufficient input, it returns a safe message (e.g., "Unable to generate reliable insight") instead of producing a potentially incorrect recommendation.
- **Overall Evaluation:**
  The AI Output & Boundary Testing demonstrates that SmartStock is capable of:
  - Generating **accurate and context-aware recommendations**
  - Handling **large and complex inputs efficiently**
  - Preventing **invalid or misleading outputs under edge conditions**
  - Maintaining **system reliability despite AI limitations**

These results confirm that the Z.AI GLM is effectively integrated as a **robust decision intelligence engine**, meeting the functional and safety requirements of the system.